

Software Categories — Detailed Lecture Notes

JKSSB Computer Awareness

October 8, 2026

18.1 What is software?

A computer system has two parts: hardware and software. **Hardware** is the physical part that you can see and touch, such as the CPU, memory, and input/output devices. **Software** is the set of programs, that is, the instructions that tell the computer what to do. Programs are stored in memory and executed by the CPU, and without software the hardware cannot perform any useful task. Where hardware is the body of the computer, software is the set of instructions that gives it purpose.

Definition (Software). *A set of programs or instructions that directs the computer to perform a task. Hardware is the physical equipment; software is the logical instructions that run on it.*

Software does not exist as a physical object; it exists as information. A programmer writes the instructions in a programming language, and these instructions are then translated, stored in memory, and executed by the CPU. The same piece of hardware — a desktop computer, for example — can behave completely differently depending on which software is loaded onto it. This is why software is described as the logical part of a computer system: it supplies the behaviour that the physical parts only make possible.

Point	Hardware	Software
Nature	Physical; can be touched	Logical; a set of instructions
Made of	Electronic and mechanical components	Programs and data
Example	CPU, RAM, keyboard, monitor	Operating system, MS Word, a browser
Failure	Fails due to wear, damage, or age	Fails due to errors (bugs) or corruption

Software is commonly grouped by *what it is for* into six categories: system software, application software, utility software, programming software, embedded software, and middleware. These categories describe purpose and they can overlap in practice, but each answers a different question about a program's role.

Category	Role in one line
System software	Controls the hardware and provides a platform for other software.
Application software	Helps the user perform a specific task.
Utility software	Performs maintenance and housekeeping tasks.
Programming software	Used to write, translate, and test programs.
Embedded software	Built into a device to control that device.
Middleware	Connects different applications and systems.

18.2 System software

System software manages and controls the computer hardware and provides the *platform* on which application software runs. It works close to the machine, usually starts when the computer is switched on, and is not written for one user's task. Common examples are the operating system, device drivers, firmware such as the BIOS, and utility programs.

The **operating system (OS)** is the most important system software. It acts as the interface between the user and the hardware and manages memory, files, processes, and input/output devices. Windows, Linux, macOS, and Android are familiar operating systems. Its main functions are process management (starting, scheduling, and stopping programs), memory management (allocating and freeing main memory), file management (creating, reading, and organising files), device management (controlling hardware through drivers), and providing a user interface. **Firmware** such as the BIOS or UEFI also belongs to the system-software layer: it initialises the hardware when the machine is switched on and then loads the operating system.

A **device driver** is also system software: it allows the operating system to communicate with a particular hardware device. It is easy to mistake a device driver for hardware, but it is a piece of software (TRAP-10).

Definition (System software). *Software that controls and coordinates the hardware and provides the services and platform that other programs need in order to run.*

18.3 Application software

Application software helps the user perform a specific task. It runs on top of the system software and is written for the user's work rather than to manage the machine. Word processors, spreadsheets, web browsers, media players, and accounting packages are all application software.

Application software is often divided into two kinds. **General-purpose** software is a ready-made package sold to many users, such as MS Word or MS Excel; it is also called an *application package*. **Specific-purpose** software, also called *tailor-made* or custom software, is built for one organisation or one job, such as a payroll system or a railway reservation system.

Application software covers many kinds of user work. Word processors prepare documents; spreadsheets handle calculations and tables; presentation tools build slide shows; database packages store and query records; graphics and multimedia packages edit images, audio, and video; browsers and email clients handle communication; and accounting packages keep business records. Most modern application software has a graphical user interface (GUI), so the user works with windows, icons, and menus instead of typing commands.

Definition (Application software). *Software that is written to help the user perform a specific task or set of tasks, running on top of the system software rather than managing the machine.*

Example 18.3.1. *The office desktop runs MS Windows and MS Office. Windows is system software because it manages the hardware; MS Word and MS Excel are application software because they help the user create documents and spreadsheets. The same machine shows all the main categories in use at once.*

18.4 Utility software

Utility software is a small program that performs maintenance or housekeeping tasks on the system. Antivirus programs, disk cleanup tools, disk defragmentation tools, backup utilities, and file-compression tools are common examples. Utility software keeps the system healthy, fast, and safe, and it belongs to the system software family.

A simple way to keep utilities and applications apart is to ask what the program is for. A utility *maintains the system* (for example, scanning for viruses or cleaning temporary files), whereas an application helps the user do *work* (for example, typing a letter or preparing a budget).

Definition (Utility software). *A small program within the system software family that carries out maintenance and housekeeping tasks such as virus scanning, disk cleanup, defragmentation, backup, and compression.*

Example 18.4.1. *A user notices that the office desktop has become slow and that the free disk space keeps shrinking. Running a disk-cleanup tool to remove temporary files and a disk-defragmentation tool to reorganise the hard disk restores speed and space. Both tools are utility software: they maintain the system rather than help the user produce a document.*

18.5 Programming software and language translators

Programming software is used to write, translate, and test computer programs. The core members are the **language translators**: a **compiler**, an **interpreter**, and an **assembler**. A compiler translates a whole program into machine code at once, while an interpreter translates and runs the program line by line. An assembler translates assembly language into machine code.

Translator	How it works
Compiler	Translates the whole program into machine code before it runs.
Interpreter	Translates and executes the program one line at a time.
Assembler	Translates assembly language (mnemonic codes) into machine code.

Because a compiler translates the whole program first, it reports errors together and the translated program generally runs faster; because an interpreter works line by line, it can start running immediately and is handy for testing small pieces of code.

Around the translators sit other supporting tools: the **linker**, which joins separate pieces of a program; the **loader**, which brings a program into memory to run; the **debugger**, which helps find errors; and the text editor used to write the code. An **IDE** (Integrated Development Environment) bundles an editor, a compiler or interpreter, and a debugger into one package, which makes programming more convenient.

18.6 Embedded software

Embedded software is built into a device in order to control that device. It is usually stored in read-only memory as **firmware**, runs automatically when the device is switched on, and is written for one dedicated job. Washing machines, microwave ovens, network routers, and the engine-control units in modern cars all contain embedded software. The user normally never sees this software directly; it simply makes the device behave correctly.

Definition (Embedded software). *Software built into a device and stored as firmware to control the device's fixed, dedicated functions.*

18.7 Middleware

Middleware is software that connects different applications and systems so that they can work together. It sits between the operating system and the application software and provides common services such as messaging, authentication, and data exchange. An application server that links a web front end to a back-end database is a typical example. Middleware allows programs that were built separately to communicate without being rewritten for each other.

Definition (Middleware). *Software that sits between the operating system and applications and connects different programs or systems, providing common services such as messaging, authentication, and data exchange.*

Middleware is common in large or distributed systems. On a web application, for example, the browser (client), the web server, and the database are three different programs. Middleware passing requests and replies between them lets these separate programs behave as one service, without each program needing to know the internal details of the others.

18.8 Software licensing models

A software **licence** states how a program may be used, copied, modified, and distributed. The main models are summarised below.

Model	Key point
Proprietary	Source code is kept closed; use is governed by the vendor's licence.
Open-source	Source code is available to view, modify, and redistribute under its licence.
Free software	Gives the freedom to run, study, modify, and share the software.
Freeware	Distributed free of cost, but the source code may not be available.
Shareware	Offered as a limited trial; payment is needed for full or continued use.
Public domain	Has no copyright and may be used freely for any purpose.
Commercial	Sold for profit; usually proprietary.

Proprietary software keeps its source code secret and restricts copying and modification. MS Windows and MS Office are examples. **Open-source software** publishes its source code and its licence allows users to view, change, and share it; Linux, Mozilla Firefox, and LibreOffice are

examples. Note that “open-source” refers to *source-code availability*, not to price. Common open-source licences include the **GNU General Public License (GPL)**, the **Apache License**, and the **MIT License**; each sets out the conditions under which the source code may be used and changed.

Proprietary software is usually supplied under an **EULA** (End User License Agreement), which the user accepts before installing the product and which forbids copying, modifying, or redistributing the program. Because the source code is closed, only the vendor can improve or fix the software.

Free software is a related but distinct idea: it refers to freedom, not to price. Free software gives the user the freedom to run the program, study and modify it, and redistribute copies. It is important to see that both free software and open-source software make the source code available; they differ mainly in the philosophy they express, with free software emphasising the user’s freedom. **Freeware**, by contrast, simply means free of cost; its source code may remain closed. **Shareware** is a trial version distributed free for a limited period, after which the user must pay for continued or full use. **Public-domain software** carries no copyright at all and may be used freely. **Commercial software** is sold for profit; it is usually also proprietary, although a commercial product can occasionally be open-source.

Definition (Open-source vs freeware vs shareware). *Open-source means the source code is available; freeware means the software is free of cost (source may be closed); shareware is a trial that must later be paid for. The three are not the same.*

Example 18.8.1. *An item states, “Freeware, open-source, and shareware all mean that the source code is freely available.” This is wrong. Only open-source guarantees source-code availability. Freeware is free of cost but may be closed-source, and shareware is a trial that later requires payment. The wrong option has merged three different licensing models (TRAP-23).*

18.9 System software vs application software

The two broadest groups are worth comparing side by side, because many exam items turn on exactly this distinction.

Point	System software	Application software
Purpose	Runs and manages the machine	Helps the user do a task
Runs	Close to the hardware; starts at boot	On top of the system software
Needed for	The computer to work at all	Performing the user’s work
Examples	Operating system, device driver, BIOS	MS Word, MS Excel, a web browser

The same physical computer can run one operating system (system software) and many applications at the same time. The operating system is what makes the hardware usable; the applications are what the user actually works with.

18.10 Exam retrieval and common confusions

Five distinctions prevent most errors on this topic.

1. **System vs application:** system software manages the hardware; application software helps the user perform a task. The operating system is system software, while MS Word is application software.
2. **Utility vs application:** a utility *maintains* the system (antivirus, disk cleanup); an application helps the user do *work*.
3. **A device driver is software**, not hardware; the operating system is system software (TRAP-10).
4. **Open-source $\neq$ freeware $\neq$ shareware** (TRAP-23): open-source gives the source code, freeware is free of cost, and shareware is a paid trial.
5. **Free software is about freedom, not price;** it should not be confused with freeware, which is about price alone.

Example 18.10.1. *An item asks which of a compiler, an antivirus program, MS Excel, and the operating system is not system software. Trace the categories: a compiler is programming software (system family), an antivirus program is utility software (system family), and the operating system is system software. MS Excel is application software, so it is the odd one out.*

Exercises

1. Define software and distinguish it from hardware.
2. List the six main categories of software and give one example of each.
3. Distinguish system software from application software, with one example of each.
4. Explain the difference between utility software and application software.
5. Distinguish open-source software, freeware, and shareware, and state which one guarantees access to the source code.
6. Classify each of the following as system, application, utility, programming, embedded, or middleware software: MS Word; an antivirus program; a compiler; the firmware in a router; an application server.

Solutions

1. Software is a set of programs (instructions) that tells the computer what to do; hardware is the physical equipment that runs those instructions.
2. System software (operating system), application software (MS Word), utility software (antivirus), programming software (compiler), embedded software (washing-machine firmware), and middleware (application server).
3. System software manages and controls the hardware and provides a platform; application software helps the user perform a specific task. The operating system is system software; MS Excel is application software.
4. Utility software performs maintenance and housekeeping tasks such as virus scanning or disk cleanup; application software helps the user carry out work such as writing a document or preparing a budget.

5. Open-source software publishes its source code and allows it to be viewed, modified, and shared; freeware is free of cost but may keep its source code closed; shareware is a trial that later requires payment. Only open-source software guarantees access to the source code.
6. MS Word is application software; an antivirus program is utility software; a compiler is programming software; the firmware in a router is embedded software; and an application server is middleware.